



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE
MÉXICO CAMPUS CULIACÁN

TAREA N REINAS

Temas de Inteligencia Artificial

Rodriguez Lopez Said Armando
Santacruz Maldonado Oscar Said

Zuriel Dathan Mora Felix

Ingeniería en Sistemas computacionales
9no Semestre

9:00 - 10:00

09/10/2026

1. Descripción del problema

El problema de las ocho reinas consiste en acomodar ocho reinas en un tablero de 8×8 sin que compartan o choquen en las filas, columnas o diagonales. La búsqueda tabú lo que hace es evaluar los ataques mediante un costo y explora soluciones hasta encontrar una con costo cero o agotar las iteraciones disponibles.

El tablero se representa mediante un vector que asigna una fila distinta a la reina de cada columna. Esta representación evita los conflictos de fila y columna, por lo que solo deben comprobarse los ataques diagonales.

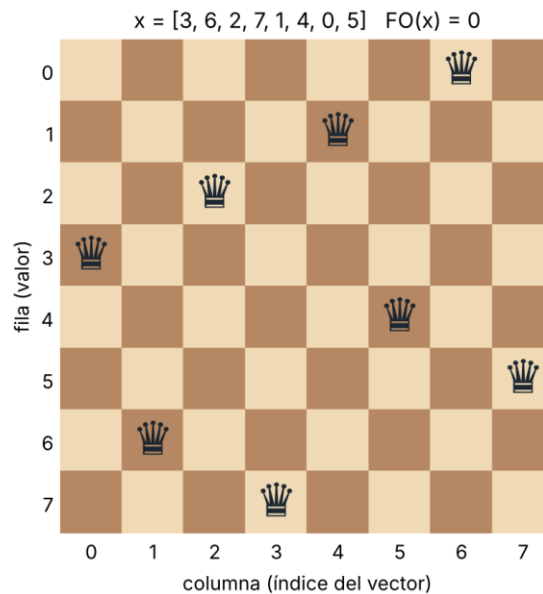


Figura 1. Configuración válida con $FO(x) = 0$.

Durante la exploración, el algoritmo acepta el mejor vecino admisible incluso si tiene más conflictos que el estado actual. Esto puede permitir salir de un mínimo local cuando todos los vecinos disponibles empeoran el costo. La lista tabú bloquea intercambios recientes durante varias iteraciones y reduce algunas repeticiones, pero no elimina todos los ciclos ni garantiza una solución dentro del límite.

2. Representación de P y S

Problema y función objetivo

P es el problema de colocar las ocho reinas sin ataques mutuos. Para resolverlo, se busca minimizar la función objetivo $FO(x)$, que cuenta los pares de reinas que comparten diagonal. Como la representación mediante vectores ya impide los conflictos de fila y columna, $FO(x) = 0$ indica una solución válida.

Las reinas de las columnas i y j , con $i < j$, comparten diagonal si la distancia entre sus filas es igual a la distancia entre sus columnas. La condición es:

$$|x[i] - x[j]| = j - i$$

$$FO(x) = |\{(i, j) : 0 \leq i < j \leq 7, |x[i] - x[j]| = j - i\}|$$

$FO(\text{tablero})$ recorre los pares mediante dos ciclos anidados y evalúa cada uno una sola vez al considerar únicamente $i < j$. Como entre ocho reinas hay 28 pares, el costo puede tomar valores de 0 a 28.

Por ejemplo, el vector $[3, 0, 4, 7, 6, 2, 5, 1]$ tiene tres pares de columnas en conflicto: $(1, 6)$, $(2, 4)$ y $(3, 4)$. En cada par, las distancias entre filas y entre columnas coinciden y valen 5, 2 y 1, respectivamente, por lo que $FO(x) = 3$. Se cuentan pares en conflicto, de modo que una misma reina puede intervenir en varios de ellos.

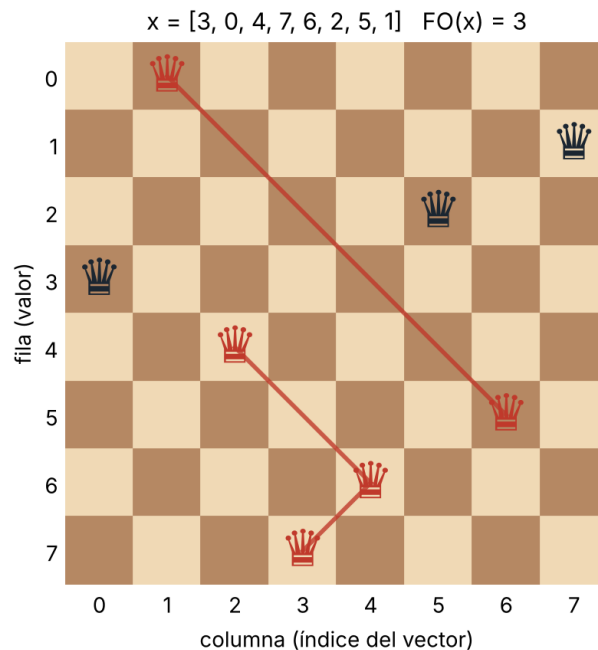


Figura 2. Los tres pares de columnas en conflicto producen $FO(x) = 3$.

Espacio de soluciones

En el vector x , el índice representa la columna y el valor almacenado representa la fila como ya se había mencionado antes, ambas numeradas desde cero, según la correspondencia $\text{tablero}[\text{columna}] = \text{fila}$. Por ejemplo, $x[0] = 3$ coloca la reina de la columna 0 en la fila 3. El vector contiene una permutación de los números del 0 al 7, de modo que cada fila se asigna una sola vez.

$$S = \{\text{permutaciones de } (0, 1, 2, 3, 4, 5, 6, 7)\}$$

$$|S| = 8! = 40\,320$$

S es el espacio de estados candidatos: incluye tanto configuraciones con ataques diagonales como soluciones válidas. El estado inicial se genera con `sample(range(num_reinas), num_reinas)`, donde `num_reinas = 8`. Las funciones toman el tamaño del valor `num_reinas` para construir un tablero $N \times N$.

Vecindario

Un vecino se obtiene intercambiando las filas asignadas a dos columnas. Para cada par $i < j$, $N(x)$ copia el vector, intercambia $x[i]$ y $x[j]$ y devuelve el vecino junto con el movimiento (swap). El intercambio conserva la permutación porque solo cambia la posición de dos valores, sin añadirlos ni repetirlos. Los $8 \times 7 / 2 = 28$ pares de columnas generan 28 vecinos distintos.

La función de buscar recorre los vecinos en el orden de $N(x)$ y sustituye el candidato seleccionado solo cuando encuentra otro aceptable con un costo estrictamente menor. Si varios vecinos aceptables comparten el costo mínimo, conserva el primero encontrado. Una vez fijados el estado inicial y los parámetros, esta selección no introduce nuevas decisiones aleatorias.

3. Propuesta de algoritmo en pseudocódigo o diagrama de flujo

Pseudocódigo

El pseudocódigo resume la selección del vecino admisible de menor costo, la actualización de la memoria tabú y la conservación del mejor estado encontrado.

Algoritmo BusquedaTabu_8Reinas

Entrada: 8 reinas, $n = 200$ iteraciones, $tt = 3$

Salida: mejor solución x^* , costo f^* e historial

```
x ← permutación aleatoria de {0,...,7}
x* ← copia de x; f* ← FO(x)
T ← lista tabú vacía; historial ← lista vacía
Para k ← 1 hasta n:
    Si f* = 0, salir del ciclo
    x' ← nulo; f' ← infinito; s ← nulo
    Para cada (v, (i,j)) en N(x):
        c ← FO(v)
        esTabu ← existe una entrada para (i,j) en T
        aspiracion ← esTabu y c < f*
        Si esTabu y no aspiracion, continuar
        Si c < f':
            x' ← v; f' ← c; s ← (i,j)
    Fin para
    Restar 1 a la vida de cada movimiento en T
    Eliminar de T las entradas con vida = 0
    Si x' ≠ nulo:
        x ← x' // aceptar aunque empeore
        Si f' < f*: x* ← copia de x'; f* ← f'
        Si tt > 0, agregar (s, tt) a T
    En caso contrario:
        s ← «sin movimiento»
    Imprimir y registrar la iteración en historial
Fin para
Mostrar el tablero x* y el reporte de métricas
Devolver x*, f* e historial
```

El estado actual x puede pasar a un vecino con mayor costo durante la exploración. En cambio, x^* conserva la mejor configuración encontrada hasta ese momento y solo se actualiza cuando se obtiene un costo estrictamente menor que f^* . Al finalizar, la búsqueda devuelve x^* , aunque el estado actual sea diferente.

Diagrama de flujo

El diagrama resume el mismo procedimiento con $\text{num_reinas} = 8$, $n = 200$ y $tt = 3$. Incluye la aspiración, la actualización de la memoria y la ruta que se sigue cuando no existe un vecino admisible.

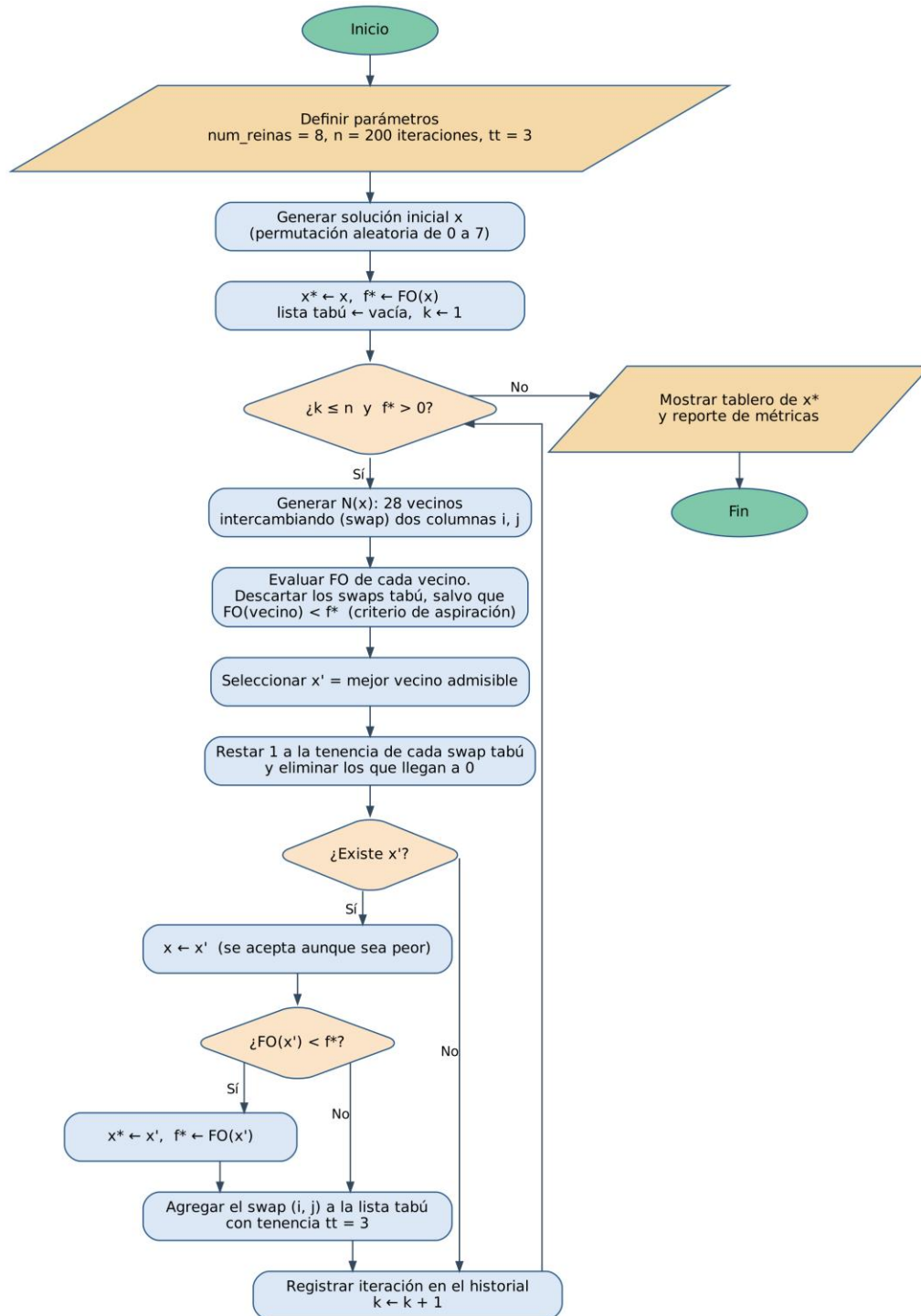


Figura 3. Flujo de la búsqueda con los parámetros del programa principal.